Case Cards: Trust Gates on Six Copilot-Agent Pull Requests

`Azure/azure-sdk-for-python`

Supplement to *Engineering Trust When AI Writes Your Software*

Evidence gathered 2026-09-28/29

Two pull requests from the case study have full walkthroughs (#41852 and #44844, separate supplements). This document gives the remaining six in a fixed one-page format: the task and change; requirements with the two classification questions and the resulting level; the gate results; the claim checks with their before/after values; the triage decision; and the claim–evidence–limitation statement. Each card is the record behind one row of Table 1 in the article. All raw outputs are in `results/evidence/`, and each card can be regenerated with `python run_experiments.py run -prs <N>`.

Levels. MUST-HOLD: violation unacceptable; strongest evidence owed. MAY-FAIL: deviation tolerable if detected. CANNOT-GUARANTEE: cannot be promised; state the limit. **Q1**: would violation be a security, safety, integrity or compliance failure, or make the artifact unfit? **Q2**: does correctness depend on an ambiguity, unobservable state, open-ended completeness, or an absent reference? Q1 yes → must-hold; else Q2 yes → cannot-guarantee; else may-fail. Both answers recorded.

Gates. G1 interface resolution, serving the standing hypothesis H0 (the change references only interfaces that exist; must-hold; attached to every PR and not repeated on the cards); G2 mutants on the changed lines vs. the package suite; G3 the same mutants vs. the PR's own tests; G4 one check per claim in the PR description, run after and before the PR.

Requirements H1, H2, ... on each card are the independent rater's (RN), extracted and classified from the PR description alone, before seeing any code or result; the reasoning column abridges his. Rows marked [a] (A1) are requirements the analyst (an AI assistant) extracted and the rater did not; they are not blind, being written after the diffs and gate results had been seen. Triage on these cards is the analyst's.

1 PR #43269 — azure-cosmos: optional `max_concurrency`; `create_database` overloads

Task and change. Two tidy-ups in one PR: make `max_concurrency` of `read_items` optional (`None` lets `ThreadPoolExecutor` choose), and align the `create_database` overloads (add `response_hook`, drop `populate_query_metrics`, make `offer_throughput` keyword-only). 140 lines across 8 files. No tests, no changelog entry. Merged 2025-10-09, commit `7e59645b2`.

The description says, twice, that the async path is unchanged: "**Async path unchanged:** ... retains `max_concurrency: int = 10`" and "# Async: Continues to use default of 10". The diff changes `aio/_read_items_helper_async.py` from `max_concurrency: int = 10` to `Optional[int] = 5` with `self.max_concurrency = max_concurrency if max_concurrency and max_concurrency > 0 else 5`.

Requirements and hypotheses.

	Q1	Q2	Level	Reasoning
H1 sync SDK uses the executor default when `max_concurrency` is unspecified	no	no	MAY-FAIL H1	no security or similar problem; described well enough to test
H2 `create_database` overloads accept `response_hook`	no	no	MAY-FAIL H2	not security-related; testable against the API signature
H3 overloads drop the deprecated `populate_query_metrics`	no	no	MAY-FAIL H3	as H2
H4 `offer_throughput` keyword-only in sync overloads	no	no	MAY-FAIL H4	not security-related; easy to test by calling both ways
A1 async behaviour unchanged, as the description states [a]	yes	no	MUST-HOLD A1	the description promises it; a silent default change breaks existing callers

Gate results. G1 pass (all references resolve; BOM-prefixed files handled). G2: 9 mutants on the changed lines (the async default `5`→`6`, `and`→`or`, `>`→`>=`, the sync default string, the hook-invocation condition), **0 killed**; the offline unit tests do not reach any of the changed code. G3 not applicable: no PR tests.

G4 checks.

Check	Before	After	Claim
Async helper signature default	10	**5**	"async unchanged" not supported
Async effective value when omitted	10	**5**	same
Same, on `main` 2026-09-28	—	5	still present
`create_database` calls `response_hook(headers)`	(no hook)	yes, one argument	supported; the new docstring types it as a two-argument callable

Triage. Confirmed defect, claim–implementation divergence, severity minor: the description asserts the async path is unchanged; the code halves the async default, with no changelog. Escaped (CI green, no review comment). Reported to the project as issue #49273 on 2026-09-30; confirmed by the Cosmos maintainer, who wrote that the reduction to 5 was intended (made between the 4.14 betas and 4.14.0) but that the changelog entry had been missed and "the Copilot PR description is also wrong". The #43269 description was corrected and PR #49306 added the 4.14.0 release note and the async docstring; the issue was closed as completed by #49306. The `response_hook` docstring mismatch was acknowledged and left out of scope. Evidence gap: 0/9, no tests. Not evaluated: throughput effect of the halved default; typing checks on the overloads.

CEL. *Claim:* sync `read_items` accepts `max_concurrency=None`; `create_database` invokes `response_hook`. *Scope:* sync path; hook call with one argument. *Evidence:* static inspection of both call sites (G4); G1 pass. *Limitations:* async default changed 10→5 contrary to the description (confirmed; on `main`); no test covers any changed line (0/9); docstring/implementation mismatch on the hook signature.

2 PR #46707 — azure-confidentialledger: `assert` → explicit `ValueError` in receipt verification

Task and change. Following an internal incident (IcM link in the PR), replace every `assert` in receipt-content validation with an explicit check raising `ValueError("The receipt content is invalid.")`, and raise `ValueError` for non-EC certificates. 54 lines in one file. No tests changed. Merged 2026-05-05, commit `53dcd778`.

Requirements and hypotheses.

	Q1	Q2	Level	Reasoning
H1 `assert`s replaced with explicit checks raising `ValueError`	no	yes	CANNOT-GUARANTEE H1	the description does not say whether the change is cosmetic or security-motivated (Q1), nor when the checks fire (Q2)
A1 malformed receipts are rejected, including under `python -O` [a]	yes	no	MUST-HOLD A1	acceptance of a bad receipt defeats verification; `-O` strips `assert`

The rater, working from the four-line description, could not tell whether the change mattered for security and classified it cannot-guarantee; the requirement it protects (A1) is a security property, which only the incident reference and the code reveal.

Gate results. G1 pass. G2: 46 mutants, **42 killed** (0.91) by the 67 existing receipt tests. Survivors: L143 `"right"` string and `in`→`not in` on the right operand; L147 `"right"` string; L381 remove `raise` (non-EC certificate). G3 n/a (no PR tests).

G4 checks. Five inputs through `_validate_receipt_content` / `_load_and_verify_pem_certificate`, under `python` and `python -O`, at merge commit and parent. `grep` for `assert` in the file: 0 after the PR.

Input	Parent	Parent `-O`	PR	PR `-O`
missing `signature`	ValueError	**accepted**	ValueError	ValueError
proof element with neither `left` nor `right`	ValueError	**accepted**	ValueError	ValueError
`right` not a string	ValueError	**accepted**	ValueError	ValueError
valid right-only element	accepted	accepted	accepted	accepted
RSA (non-EC) certificate	ValueError	**accepted**	ValueError	ValueError

Triage. No defect attributed to the PR; the claim holds and closes a real gap in a security property (A1). **Two undocumented limitations**: (1) nothing in CI runs under `-O`, so the property the PR restores has no regression protection and a future revert to `assert` passes; (2) the right-sibling proof path and the non-EC certificate path are never exercised by the suite, so the PR's conversion of those checks is unverified (the four survivors). Not evaluated: the cryptographic verification itself (unchanged); live-ledger receipts.

CEL. *Claim:* receipt validation rejects the malformed classes exercised, with or without assert stripping. *Scope:* left-sibling proofs, EC certificates. *Evidence:* G4 table; 42/46 mutants killed; 67 tests green. *Limitations:* right-sibling and non-EC paths unverified; no `-O` regression test; live receipts not tested.

3 PR #41492 — azure-ai-evaluation: `int()` → `float()` in threshold comparison

Task and change. `MeteorScoreEvaluator` with threshold 0.5 and score 0.9375 reported `fail`, because `_base_eval.py` compared `int(score_value)` to the threshold. Two lines changed (`int` → `float` in both the higher-is-better and lower-is-better branches), a version bump, and 47 new test lines for Meteor and Bleu with decimal thresholds. Merged 2025-06-10, commit `0c347e994`.

Requirements and hypotheses.

	Q1	Q2	Level	Reasoning
H1 correct pass/fail for decimal scores, higher- and lower-is-better	yes	no	MUST-HOLD H1	an incorrect comparison gives incorrect results

Gate results. G1 pass. G2 = G3 (the PR's file is the only test of the branch): 5 mutants, 1 killed (L419 negate). Survivors: L419 `>=`→`>`; L424 `<=`→`<`; L424 negate (the entire lower-is-better branch); a version-string mutant (equivalent, excluded; reported as 1/4).

G4 checks.

Check	Before	After	Claim
C2 `F1ScoreEvaluator`, score 0.800 vs threshold 0.5	fail	pass	"other score-based evaluators benefit" supported
C3 synthetic lower-is-better, score 2.5 vs threshold 2	**pass (wrong)**	fail	"both branches" supported, untested by PR
C3 score 0.4 vs 0.5; score 2.0 vs 2	pass; pass	pass; pass	—
C4 score == threshold (higher-is-better)	fail	pass	boundary now correct; untested
R score `"N/A"`	swallowed	swallowed	pre-existing: `except: print()`

Triage. No defect attributed. **Two undocumented limitations**: the lower-is-better branch had the inverse bug (2.5 against threshold 2 reported `pass`) and is fixed, but no PR test exercises it and its negate mutant survives; the equality boundary is untested. Evidence gap (pre-existing): non-numeric scores are silently swallowed and produce no `_result` key. Not evaluated: content-safety evaluators against the live service; rouge keys (excluded from the loop).

CEL. *Claim:* threshold comparison uses the float score; higher-is-better evaluators pass at and above the threshold. *Scope:* numeric scores; higher-is-better branch as tested. *Evidence:* G4 C2/C4; PR tests (18); G1 pass. *Limitations:* lower-is-better fix unprotected by tests; boundary untested; non-numeric scores silently dropped.

4 PR #44770 — azure-cosmos: port of hotfix #44649 (partition-split recursion)

Task and change. Port a human-authored hotfix from a release branch to `main`. The hotfix stops an infinite recursion when a partition-key-range fetch itself hits a 410 (partition split): the routing-map provider sets a private flag `_internal_pk_range_fetch`, the request layer pops it before the HTTP call, and the execution context skips its retry loop when the flag is set; state (`_has_started`, `_continuation`) is reset on retry; logging added. Six source files, sync and async; two new unit-test files (328 + 317 lines). Merged 2026-01-22, commit `9a8a19a39`.

Requirements and hypotheses.

	Q1	Q2	Level	Reasoning
H1 partition-split handling does not recurse infinitely on a 410	yes	no	MUST-HOLD H1	infinite recursion makes the operation fail instead of handling the split
A1 the port matches the hotfix #44649 [a]	yes	no	MUST-HOLD A1	a divergent port ships an untested variant

Gate results. G1 pass. G2 = G3: 34 mutants, **8 killed** (0.24), identically by suite and PR tests. Killed: the execution-context guard in sync and async (flag key string, `False`→`True` default, negate, `return None`). Survived: every log-message string (cosmetic); `_has_started = False`→`True` in sync and async (the state reset); `return []`→`None` at the unreachable loop exit; `_internal_pk_range_fetch: True`→`False` and its key string in both routing-map providers; the key string of the `pop` in both request layers.

G4 check (port fidelity). `git fetch origin pull/44649/head:pr44649`, then `git diff pr44649 9a8a19a39` per file:

File	Difference between hotfix and port
`_execution_context/base_execution_context.py` (+ aio)	+4 lines: one `_LOGGER.warning` before the unreachable `return []` (requested in review)
`_routing/routing_map_provider.py` (+ aio)	identical
`_synchronized_request.py`, `aio/_asynchronous_request.py`	differ only by unrelated `main` drift (hedging); `kwargs.pop("_internal_pk_range_fetch", None)` present in both versions of both files

Triage. No defect; port claim supported. **Two undocumented limitations**: the state reset is untested (its mutants survive in sync and async); the flag is untested at the provider and request layers, so only the execution-context half of the anti-recursion mechanism is covered by the 645 lines of tests the PR shipped. Not evaluated: behaviour against a live account during an actual split; async test collection count.

CEL. *Claim:* the port reproduces #44649; the execution context skips retry when the internal flag is set. *Scope:* execution-context guard. *Evidence:* G4 diff; 8/34 mutants killed on the guard; PR unit tests. *Limitations:* state reset and provider/request-layer flag handling untested; live split behaviour not evaluated.

5 PR #42047 — azure-ai-evaluation: randomization seed across simulators

Task and change. Users reported that two runs of a safety evaluation with the same `randomization_seed` produced different query sets (different counts). The PR forwards the seed from `_SafetyEvaluation` to `Simulator`, replaces `random.seed()` in `AdversarialSimulator` with a local `random.Random(seed)`, and adds the parameter to `IndirectAttackSimulator` and `Simulator`. 48 new test lines. Merged 2025-07-17, commit `54d45994a`.

Requirements and hypotheses.

	Q1	Q2	Level	Reasoning
H1 the same seed always returns the same query sets	yes	no	MUST-HOLD H1	different sets defeat the seed's purpose and invalidate experiments; testable from the description
H2 a seeded run does not modify the global random state	yes	no	MUST-HOLD H2	interference with global state makes other results irreproducible

Gate results. G1 pass. G2 = G3: 5 mutants (`is not`→`is` and negate on the seed guards in two files; `and`→`or`), **0 killed**. The two new tests instantiate `random.Random(seed)`, shuffle a list twice and compare; they never construct a simulator.

G4 (static; the simulators need the RAI service).

Check	Result
Seed forwarded to `Simulator` in `_safety_evaluation.py`	yes (diff line)
Local `Random` used for template shuffling	yes; the unseeded branch still calls global `random.shuffle`
Result collection order	`asyncio.as_completed`: completion order, regardless of seed
Row-count logic touched by the PR	no: `min(total_tasks, max_simulation_results)` unchanged

Triage. Possible claim divergence, not confirmed: the description's root cause (unseeded shuffling) does not obviously explain the reported symptom (different counts), and ordering is nondeterministic by construction; only a live run can confirm or refute the "identical query sets" claim. Recorded as a limitation pending that run; if it cannot be run, the row in Table 1 should say "limitation" only. Evidence gap: 0/5; the tests exercise the standard library. Not evaluated: live simulation; determinism of returned order.

CEL. *Claim:* the seed is threaded through all three simulators and template selection uses a seeded local generator. *Scope:* template selection; not result order or count. *Evidence:* static inspection (G4); G1 pass. *Limitations:* query-set identity and the count symptom unverified; 0/5 mutants killed; global-state claim unverified at runtime.

6 PR #45366 — azure-core: `format_url` trailing slash (tooling smoke test)

Task and change. A base URL ending in `/` lost the slash when the URL template was query-only. One line: `.rstrip("/")` removed from `PipelineClientBase.format_url`; one new test reproducing the issue's exact input. Merged 2026-03-02, commit `41391f1e`.

Requirements and hypotheses.

	Q1	Q2	Level	Reasoning
H1 query-only `format_url` templates preserve the trailing slash	yes	no	MUST-HOLD H1	a changed URL can address a different resource

Gate results. G1 pass (1 changed line). G2: **0 mutants**. The changed line only deletes a call and contains no node the nine operators apply to; the mutator's lack of a statement-deletion or call-removal operator is a documented limitation. G3 n/a. G4 not run: the PR's own test is the issue's scenario and passes.

Triage. No findings. The PR is below the study's size threshold and is retained in Table 1 only as an end-to-end check of the tooling; its row makes no claim.

CEL. *Claim:* `format_url` preserves a trailing slash when the template is query-only. *Scope:* that input class. *Evidence:* PR test; G1 pass. *Limitations:* no mutation evidence (tooling limitation); other URL shapes not exercised.